LUIS ROMERO

Backend Developer | AI Engineer (Node.js/TypeScript)

Mérida, Venezuela | +58 424 709 2980 | lueduar15@gmail.com | linkedin.com/in/hernandezrs955 | github.com/ROMEROLUIS15

RESUMEN PROFESIONAL

Backend Developer / AI Engineer enfocado en llevar sistemas de IA a producción con usuarios reales. Construyo backend para una empresa de software de Texas (Estados Unidos) y antes coordiné el desarrollo de software de una institución pública, donde decidía stack, arquitectura y estándares de calidad. En paralelo opero tres sistemas en producción: Cronix, mi SaaS multi-tenant donde agentes de IA para WhatsApp, voz y llamadas gestionan citas de forma autónoma; un SaaS para un taller de motos con agentes en Python/LangGraph; y un CMMS industrial en uso diario. Mi trabajo no es integrar un LLM: es diseñar los controles que impiden que invente datos o actúe sobre el cliente equivocado, y medir su fiabilidad en vez de suponerla.

EXPERIENCIA

Backend Developer | Complexity (Texas, Estados Unidos, Remoto) Julio 2026 - Presente

Proyecto Factory: backend de una plataforma B2B de directorio de empresas y de su panel de administración, con app móvil. API REST, tiempo real por WebSockets, colas de trabajo, búsqueda full-text y agentes de IA (Node.js, NestJS, TypeScript, PostgreSQL, Redis/BullMQ, Elasticsearch, AWS).

- Audité el backend al llegar, presenté los hallazgos y me aprobaron el plan de remediación, que dirigí hasta cerrarlo sin frenar el roadmap: con refactoring incremental eliminé el acoplamiento indebido entre módulos, bajé las dependencias circulares de 31 a 0 y llevé la lógica de negocio a 330 casos de uso sobre arquitectura hexagonal.
- Diseñé el harness de agentes de IA con el que trabajo: reglas numeradas que el propio código cita, gates de arquitectura y calidad, y un control que pide evidencia de cada afirmación del agente (Orca, Gentle AI, MCP, Spec-Driven Development). Con él planifiqué el trabajo en 700+ tickets y entregué 1.050+ pull requests integrados y 20+ releases a producción en tres meses, revisando y aprobando cada cambio.
- Construí desde cero el pipeline de calidad y seguridad. Al llegar, ningún pull request ejecutaba pruebas. Hoy cada cambio pasa por tipado estricto, pruebas unitarias, de integración y end-to-end, escaneo de seguridad (SAST/SCA, secretos) y reglas de arquitectura, y un PR completo se valida en ~6 min. Las pruebas unitarias pasaron de 20 a 1.000+ suites. El chequeo de calidad bajó de 9,4 a 3,8 min, con ejecución en paralelo y suites de integración elegidas por grafo de dependencias.
- Lideré de punta a punta el backend de "Reclama tu empresa", el programa de salida al mercado (GTM), hoy en producción y alimentado por una carga asistida por IA que llevé a producción: un agente arma perfiles de empresa y borradores de producto que nadie publica sin aprobación humana. Entregué además la integración con Resend, pedidos con máquina de estados y búsqueda con Elasticsearch.
- Diseñé y construí un motor de extracción con IA con prácticas actuales de ingeniería de agentes: cada dato lleva la cita de su fuente, dos modelos deben coincidir y el código, no el modelo, decide qué se guarda; la calidad se mide con evals contra la decisión del revisor humano y trazas en Langfuse. Casi triplicó la precisión al clasificar categorías.
- Reforcé la seguridad y la operación. Llevé la seguridad a nivel de fila (RLS) de PostgreSQL a producción: de 0 a 63 tablas aplicadas, en tres olas reversibles, con pruebas pgTAP que bloquean el merge. Resolví en menos de 4 h un reporte urgente de exposición de datos personales, añadí rate limiting con Redis y operé despliegues en AWS (EC2, CloudWatch, Secrets Manager), con dos incidentes de producción cerrados en menos de una hora.

IBIME Connect (Plataforma institucional, Remoto) Enero 2025 - Julio 2026

Coordinador de Desarrollo de Software (Febrero 2026 - Julio 2026) | Backend / Full-Stack Developer (Enero 2025 - Febrero 2026)

- Reemplacé el sistema antiguo (HTML/CSS) de una red pública de bibliotecas por una plataforma digital completa, con un asistente de IA que atiende las consultas de los ciudadanos. La institución la aprobó, la adoptó como su plataforma oficial y, a raíz de esa entrega, me designó Coordinador de Desarrollo de Software.
- Dirigí el desarrollo de software de la institución: decidí el stack y la arquitectura, fijé los estándares de pruebas y calidad, y prioricé el roadmap junto a bibliotecarios y el área de informática.
- Conseguí que el asistente no inventara respuestas manteniendo al modelo fuera del flujo de decisión: un clasificador de intención por reglas (sin LLM) enruta la conversación, y si el correo del ciudadano ya existe la respuesta sale de la base de datos sin llamar al modelo. Una cadena determinista (intención, protección de datos, umbral de coincidencia, revisión y política de salida) valida antes de responder.
- Implementé búsqueda semántica sobre los documentos de la institución (RAG con PostgreSQL + pgvector y embeddings de Gemini de 768 dimensiones, con inferencia en Groq gpt-oss-20b configurable por entorno y caché en Redis), y un agente LangGraph que lee los PDFs del catálogo y los convierte en datos estructurados y validados, descartando duplicados.
- Endurecí la plataforma para tratar datos personales de ciudadanos: bloqueé la escritura anónima sobre las tablas con datos sensibles (RLS), evité inscripciones duplicadas y definí cuánto tiempo se conserva cada dato y cómo se borra. Arquitectura hexagonal con inyección de dependencias (tsyringe), logs estructurados (Pino) y captura de errores (Sentry); 420+ tests en CI (Vitest + Playwright) y control de dependencias.

Backend Developer | Kiura (Barranquilla, Colombia, Remoto) Abril 2024 - Diciembre 2024

- Construí los servicios backend de una plataforma de teleterapia: el pago de la suscripción activa o suspende el acceso del paciente automáticamente (MercadoPago vía webhook), con chat en tiempo real (Socket.IO) y videosesiones con acceso firmado por sala (Agora).
- Implementé doble autenticación (JWT + Google OAuth) con permisos por rol sobre un dominio de 26 entidades en MySQL/Sequelize, con agenda y detección de choques de horario; desplegado en Render con base de datos en AWS (RDS).

PROYECTOS

Fundador y Backend / AI Engineer | Cronix (en producción; operación y mantenimiento) Enero 2026 - Presente

SaaS multi-tenant para negocios con citas, construido y operado solo (BD, backend serverless, IA e infraestructura). En producción 24/7: 170+ citas reales gestionadas por el agente y recordatorios automáticos, sin intervención manual. 1.600+ tests (Vitest, Playwright, pgTAP) y CI/CD con Sentry.

- Construí los agentes de WhatsApp y voz que agendan, reagendan y cancelan citas solos: el cliente lo pide hablando, a cualquier hora, y nadie del negocio tiene que intervenir. Motor de orquestación propio (sin framework), con rutas programadas que resuelven la mayoría de los casos sin llamar al modelo, y acceso restringido a herramientas (Groq + Deepgram, 12 capacidades).
- Construí un agente de llamadas en tiempo real que atiende por WhatsApp y agenda citas en Cronix hablando (Python, Pipecat Flows, WebRTC, Deepgram): el modelo conversa y el código decide cada dato y cada escritura, incluida la confirmación del cliente. Del lado de Cronix, una API firmada (HMAC) con idempotencia y un bloqueo que impide reservar dos veces el mismo horario.
- Elegí el modelo del agente de llamadas midiendo, no por intuición: en el dataset golden obtuvo 1,00 en no inventar datos y en no escribir sin pedido, el primer token llega en 391 ms (p50) y cada llamada queda trazada en Sentry con su latencia y su costo.
- Evité que el agente inventara datos o actuara sobre el cliente equivocado —el riesgo real de darle a un LLM permiso de escritura sobre una base de datos con clientes reales— con 6 capas de control, entre ellas una confirmación obligatoria en dos pasos antes de escribir y un segundo modelo de IA que revisa la operación y puede vetarla (LLM-as-judge).
- Construí el sistema que comprueba si el agente responde bien: guardé conversaciones reales como referencia y las vuelvo a ejecutar en cada cambio; el CI bloquea el merge si el agente empeora (conversaciones golden), y aparte lo mido con DeepEval y un juez LLM (relevancia y alucinación). Cada conversación queda registrada con su latencia (p50/p95) por negocio, en un panel propio (PostgreSQL + LangSmith) sobre 9 Edge Functions (Deno).
- Audité mi propia seguridad hasta encontrarle un agujero: descubrí que 13 funciones de la base de datos permitían ver datos de otros negocios, reproduje el ataque para confirmarlo, lo cerré con una validación de pertenencia y lo dejé cubierto por 147 pruebas automáticas de seguridad (pgTAP sobre 59 políticas de aislamiento RLS). También medí cuánto aguanta antes de que lo hiciera un cliente: con 500 negocios y 600.000 citas cargadas, el panel sigue respondiendo en menos de 17 ms.
- Integré 4 métodos de pago (PayPal, NOWPayments, Pago Móvil, Binance) a prueba de cobros duplicados: aunque el proveedor notifique dos veces el mismo pago, al negocio se le cobra una sola vez.

Backend / Full-Stack Developer | Motosmax Cordialidad (Barranquilla, Colombia, Remoto) (entregado por fases; en producción con soporte) Junio 2026 - Presente

- Puse en producción el SaaS con el que un taller de motos trabaja a diario: órdenes de trabajo con máquina de estados, inventario por sucursal, cotizaciones versionadas con aprobación del cliente, venta de motocicletas y ficha de cliente. Monorepo con API en NestJS (arquitectura hexagonal, 37 modelos de datos en Prisma) y frontend en Next.js 15.
- Automaticé la atención al cliente y a la administración con un microservicio en Python (FastAPI + LangGraph): un agente atiende por WhatsApp a los clientes y otro asiste a la administración con memoria de sesión, reportes periódicos y aviso de stock bajo. El modelo LLM (Groq/DeepSeek) es configurable por entorno, y el canal de WhatsApp maneja la ventana de 24h de Meta y reporta los fallos de envío.
- Blindé la comunicación entre servicios con credenciales que caducan a los 5 minutos y cifrado de campos sensibles; colas de trabajo (BullMQ/Redis), almacenamiento de fotos y PDFs en Cloudflare R2 y un gateway WebSocket para tiempo real. 400+ tests (Jest, pytest + E2E con Playwright) con lint y tipado estricto en CI; los use-cases no acceden directo a Prisma, solo por puertos y repositorios. Cada cambio arranca de una especificación versionada (Spec-Driven Development).
- Encontré y cerré una escalada de privilegios en producción: un rol de solo lectura podía cambiar la configuración del taller (el IVA y los datos impresos en cotizaciones y contratos); reproduje el fallo, lo corregí con permisos por rol y lo blindé con un test que impide que reaparezca. Además pasé el límite de peticiones a identificarse por usuario, no por IP compartida.

CMMS Hidrobombas Mérida (Cliente, Remoto) | Backend / Full-Stack Developer Abril 2026 - Presente

(entregado por fases; en producción con soporte)

- Saqué del papel a una empresa de mantenimiento industrial: construí el CMMS (52 endpoints REST) que reemplazó su hoja de inspección impresa y que hoy usan todos los días, con los requerimientos levantados directamente con el cliente en entregas iterativas. Medí su techo real antes de prometer nada: con 200 usuarios simultáneos (k6) aguanta sin un solo error de servidor (p95 276 ms), y el límite que encontré —los reportes se numeran de uno en uno para que no se repita el número— decidí no tocarlo, porque quitar ese candado devolvería reportes duplicados y el techo está muy por encima del uso real del cliente.
- Construí su asistente de IA, que responde preguntas consultando el historial real de reportes en vez de improvisar (LangGraph + RAG persistente sobre pgvector), con la capa de IA aislada para poder probarla sin depender del proveedor de turno. 394 tests, 82% de cobertura en backend, con la CI corriendo la suite también contra PostgreSQL real (no solo SQLite).

Advanced Product Search API (Reto técnico con el que entré a Complexity; desplegado y público) | Backend Developer Julio 2026

API de búsqueda de productos/fabricantes con relevancia, filtros, faceting, autocompletado y sugerencias.
En vivo: advanced-search-api-chet.onrender.com/docs · github.com/ROMEROLUIS15/advanced-search-api

- Construí el buscador que ordena los resultados por lo que la persona realmente busca y no por orden de llegada: relevancia BM25 sobre Elasticsearch 8 con tolerancia a errores de tipeo, ajustada por popularidad y por qué tan reciente es el producto (function_score), con filtros combinables por categoría, subcategoría, ubicación y precio, autocompletado mientras se escribe y sugerencias "quisiste decir".
- Resolví el filtrado por facetas sin encerrar al usuario en su propia selección: los contadores de cada dimensión se calculan excluyendo su propio filtro, así puede ampliar la búsqueda en vez de quedarse sin resultados; resultados, facetas y sugerencias salen en una sola consulta a Elasticsearch. Arquitectura hexagonal (dominio sin framework, Elasticsearch y Redis detrás de

puertos), caché que nunca tumba una petición si Redis se cae y clave de API en todos los endpoints de cliente. Probé el límite de peticiones bajo ráfaga: corta exacto en el tope, responde 429 y no devuelve un solo error de servidor.

- Lo entregué desplegado y verificado, no "funciona en mi máquina": Docker + Render/Elastic Cloud/Upstash con contrato OpenAPI público, 500+ pruebas (unitarias, e2e e integración contra Elasticsearch real, 97% de cobertura), prueba de carga k6 (398.000 peticiones, 0 fallos) y una cadena de seguridad automatizada en CI que revisa el código, las dependencias, los secretos de todo el historial, la imagen que se despliega y la API en ejecución (CodeQL, Dependabot, gitleaks, Trivy + SBOM y OWASP ZAP: 128 URLs, 0 hallazgos).
- Resolví el cambio de estructura del buscador sin sacar la API de servicio: cada versión del índice se nombra por la huella de su definición y el cambio se activa moviendo un alias de forma atómica, así nadie llega a ver el índice a medio construir. Separé además la señal de "el proceso vive" de la de "está listo para recibir tráfico" —el chequeo de la plataforma dejó de consultar Redis en cada ping— y el despliegue solo corre si la CI de ese commit pasó, con trazas y métricas expuestas vía OpenTelemetry.

HABILIDADES TÉCNICAS

Backend: Node.js, TypeScript, JavaScript, Python, NestJS, Express, FastAPI, Pydantic, asyncio, uv, REST APIs, OpenAPI/Swagger, Email transaccional (Resend, React Email), Socket.IO, Serverless, Edge Functions, Deno, Microservicios, Event-Driven Architecture, Eventos de dominio, Unit of Work, Webhooks, Idempotencia, SOLID, Refactoring, Deuda técnica, Inyección de Dependencias, Arquitectura Hexagonal, Clean Architecture, Architecture Decision Records (ADRs), WebSockets, SSE, C#/.NET (base)

IA: LLM Integration, LangChain, LangGraph, AI Agents, Multi-Agent Orchestration, Tool Calling / Function Calling, Structured Outputs, Prompt Engineering, Agentic AI, Vercel AI SDK, RAG (Retrieval-Augmented Generation), Semantic Search, Vector Databases (pgvector), Anti-Hallucination, AI Guardrails, LLM Evaluation (DeepEval), LLM Observability (LangSmith, Langfuse), MCP, Voice AI, Pipecat, Pipecat Flows, WebRTC, Speech-to-Text, Text-to-Speech, OpenRouter, OpenAI API, Groq, DeepSeek, Gemini, Deepgram, n8n

Harness Engineering y desarrollo asistido por IA: Claude Code (Anthropic), Codex (OpenAI), Orca (ADE), Gentle AI, Engram (memoria persistente de agentes), MCP (Linear, Sentry, AWS, Resend, Figma), orquestación multi-agente, subagentes, hooks y skills para agentes, Context Engineering

Datos: PostgreSQL, SQL, Prisma, TypeORM, Supabase, Neon, AWS (RDS), MySQL, Sequelize, SQLite, Redis, BullMQ, pgvector, Elasticsearch, Búsqueda full-text y relevancia (BM25), Faceting, Row-Level Security, Multi-tenant

Seguridad y pagos: JWT, OAuth 2.0, WebAuthn/Passkeys, RBAC, Rate Limiting, HMAC, Threat Modeling, Auditoría de seguridad, PayPal, MercadoPago, QStash, Colas y procesamiento asíncrono

DevOps y testing: Docker, AWS (S3, EC2 Auto Scaling, ECR, CloudWatch, Secrets Manager), Git, GitHub Actions, Husky, Linear, CI/CD (Continuous Integration / Continuous Deployment), Vercel, Render, Cloudflare (Pages/R2), Sentry, Observabilidad, OpenTelemetry, Prometheus, SAST/DAST/SCA (CodeQL, Semgrep, OWASP ZAP, Trivy, gitleaks, Dependabot), SBOM, Seguridad de la cadena de suministro, Load Testing (k6), Vitest, Jest, pytest, Hypothesis (property-based testing), Snapshot testing, mypy (strict), Ruff, Bandit, pip-audit, Playwright, pgTAP, Spec-Driven Development (SDD)

Frontend: React, Next.js 15, Server Actions, Tailwind CSS, Zod, PWA, i18n

EDUCACIÓN

Ingeniería en Sistemas Informáticos | UPTM Kléber Ramírez - Graduado: Oct 2026

Bootcamp Full-Stack | Academlo - Mar 2024

IDIOMAS

Español: Nativo | Inglés: Intermedio (B1+), lectura y escritura técnica avanzada